Grid layout

Grid layout
- 1.Intro
- 2.Conceptos básicos
- 3.Creación de la cuadrícula
  - Definición de un elemento como contenedor de cuadrícula
  - Definición de columnas y filas
  - MINMAX
  - REPEAT (nº, tamaño)
  - Auto-fill
  - Auto-fit
  - Altura(FILAS):
  - Nombres de las líneas de cuadrícula
  - Separación entre columnas o entre filas
- 4.Cuadrícula implícita
  - 4.1 Algoritmo de posicionamiento automático
  - 4.2 Tamaño de las filas y columnas creadas automáticamente
- 5.Posicionamiento de elementos de cuadrícula
- 6. Áreas de cuadrícula
  - Definición de áreas
  - Posicionamiento de un elemento en un área
- 7.Propiedades de alineación de elementos en las celdas
  - Alineación horizontal
  - Alineación vertical
- 8.Propiedades de alineación de la cuadrícula respecto al contenedor
  - Alineación horizontal
  - Alineación vertical
- Demo de cuadrícula implícita y alineación
- Demo de áreas de cuadrícula y posicionamiento
- Generadores de cuadrículas
- Referencias

1.Intro

Una de las **principales tareas del diseño web** es la organización de la **disposición** (posicionamiento) **de los distintos elementos dentro de la página web, a lo que se conoce como diseño del *layout*.** El problema es que CSS no fue diseñado inicialmente pensando en el diseño de *layouts*, por lo que los desarrolladores han tenido que emplear para tal fin distintas técnicas generales no creadas específicamente para tal fin. Primero se usaron elementos `<table>`, después elementos flotantes con la propiedad CSS `float`, luego el valor `inline-block` de la propiedad `display` y últimamente `flexbox`. Todas ellas cumplen su función siempre y cuando son empleadas correctamente, pero complican bastante la tarea al desarrollador, dado que no fueron creadas específicamente para el diseño de *layouts*.

Sin embargo, recientemente se ha estandarizado un nuevo módulo de CSS, creado específicamente para el diseño de *layouts*, conocido como CSS Grid Layout, que permite diseñar interfaces de usuario CSS basadas en el posicionamiento de elementos dentro en una rejilla (*grid*) bidimensional.

Después, para colocar un elemento hijo directo del contenedor en una determinada celda de la cuadrícula usaremos en dicho hijo las propiedades `grid-column` y `grid-row`. En este sentido, al igual que en el caso de flexbox, el orden en el que aparezcan los hijos en el documento HTML es totalmente indiferente, lo que nos permitirá cambiar de manera sencilla la disposición de los elementos dependiendo del tamaño disponible para visualización.

2.Conceptos básicos

Para poder usar correctamente este módulo CSS es necesario conocer antes los términos que emplearemos:

- **Contenedor de la cuadrícula** (*grid container*): Elemento en cuyo interior se crea la cuadrícula. Para hacer que un elemento actúe como contenedor de cuadrícula debemos establecer en él la propiedad `display: grid` (elemento en bloque que en su interior actúa como contenedor de cuadrícula), `display: inline-grid` (elemento en línea que en su interior actúa como contenedor de cuadrícula).
- **Elementos de cuadrícula** (*grid items*): **Cada uno de los hijos directos** del contenedor de la cuadrícula, que podran ser posicionados en una celda concreta de la misma.
- **Líneas de cuadrícula** (*grid lines*): Cada una de las **lineas de division que componen la estructura de la cuadrícula**. Pueden ser verticales u horizontales, dependiendo de si separan, respectivamente, columnas o filas.
- **Columnas o filas de cuadrícula** (*grid tracks*): Cada uno de los espacios entre dos líneas de cuadrícula adyacentes.
- **Celdas de cuadrícula (*grid cell*):** Cada uno de los espacios entre dos líneas de cuadrícula de filas adyacentes y dos líneas de cuadrícula de columnas adyacentes. Es la "unidad" principal de posicionamiento de elementos dentro de la cuadrícula.
- **Área de cuadrícula (*grid area*)**: Espacio total rodeado por cuatro líneas de cuadrícula, **de forma rectangular** y que puede comprender más de una celda.

3.Creación de la cuadrícula

Definición de un elemento como contenedor de cuadrícula

La creación de una cuadrícula (rejilla) es bastante sencilla, consiste basicamente en designar un determinado elemento como contenedor de la cuadrícula, para lo que establecemos su propiedad `display: grid`. Por ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de definición de un elemento como contenedor de cuadrícula

Por defecto, el elemento se convierte en una cuadrícula con una única columna. Luego se trata de posicionar cada hijo, creando las filas que sean necesarias. En el ejemplo anterior se ha añadido la propiedad `grid-gap` para añadir una separación (*gutter*) de `10px` entre cada dos filas o entre cada dos columnas, sólo con el propósito de que veamos más claramente qué filas y columnas se han creado.

Definición de columnas y filas

Los normal, sin embargo, es que queremos indicar, al menos, el tamaño de las columnas. La propiedad `grid-template-columns` permite definir **en el contenedor de la cuadrícula** la anchura de cada una de las columnas de la misma, mientras que la propiedad `grid-template-rows` permite definir la altura de cada una de las filas.

Propiedad	**grid-teplate-columns, grid-template-rows**
Valores	[nombre-linea] tamaño-trazo... | none
Se aplica a	Contenedores de cuadrícula
Valor inicial	none
Descripción	Define el nombre de las líneas de cuadrícula y el tamaño de las columnas / filas de la cuadrícula.

Los **valores se especifican como una lista de tamaños separados por espacio en blanco.** Para especificar el tamaño de cada columna / fila tenemos distintas opciones.

Una **primera opción** es definir una columna o fila de un tamaño fijo, como por ejemplo `50px`, expresado en cualquier unidad válida de las que ya conocemos. En el siguiente ejemplo indicamos que la cuadrícula debe tener dos columnas de `200px` cada una:

```
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: 200px 200px;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de columnas de tamaño fijo

TAMAÑO FIJO: En el ejemplo anterior, a la hora de colocar el primer hijo se ha creado una primera fila con las dos columnas especificadas, de manera que el **primer hijo se ha colocado en la primera celda y el segundo hijo en la segunda celda de dicha fila**. A la hora de colocar el tercer hijo se ha creado una nueva fila y así sucesivamente.

En dicho ejemplo, la cuadrícula siempre medirá `410px` de ancho, independientemente de la anchura del elemento padre, por lo que si éste tiene una anchura mayor, el espacio sobrante no es utilizado por la cuadrícula, mientras que si el elemento padre tiene una anchura menor, la cuadrícula sobresaldrá del elemento (a no ser que modifiquemos el valor de la propiedad `overflow`).

Para indicar el ancho de las columnas o la altura de las filas también podemos usar un tamaño especificado en `%`, que se calcula en relación a la anchura o altura del contenedor. Por ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: 40% 40%;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de anchura de columnas en porcentaje

TAMAÑO %: En el ejemplo anterior, cada columnas tiene una anchura correspondiente al `40%` de la anchura del padre, de manera que el espacio sobrante en el padre) `20%` de anchura del padre - `10px` del gutter) no es usado.

AUTO: Algunas veces, lo que queremos es que una columna ocupe todo el ancho que pueda, de manera que la cuadrícula ocupe toda la anchura del padre. Para ello podemos usar el valor `auto`. Por ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: auto auto;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de columnas con anchura auto

En el ejemplo anterior toda la anchura del contenedor se reparte proporcionalmente entre las dos columnas, ya que ambas han especificado el valor `auto`. Dicho espacio sobrante es calculado una vez se han tenido en cuenta el resto de tamaños fijos de columnas o filas especificadas y la separación especificada entre columnas o filas.

Si no queremos que se reparta equitativamente la anchura sobrante del contenedor, sino que **queremos que una columna reciba más que la otra, podemos especificar el tamaño usando la unidad** `fr`, como por ejemplo `1fr`, donde `fr` hace referencia a una fracción de espacio libre restante disponible en el contenedor. La magnitud que acompaña a la unidad `fr` indica la proporción de dicho espacio restante que se le debe dar a esa columna, en relación a la suma de todas las magnitudes de las columnas que usan la unidad `fr`. Por ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: 2fr 1fr;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de columnas con anchura especificada en la unidad fr

FR+AUTO: Si en la definición de la anchura de las columnas mezclamos el uso de la unidad `fr` con el valor `auto`, éste último adquiere el significado de la anchura necesaria para mostrar los elementos correspondientes a celdas de dicha columnas. Por ejemplo:

¿Por qué es tan ancho si para mostrar el texto item1 necesita menos anchura?

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: auto 1fr;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
.item:first-child {
  width: 200px;
}
```

> Demo de mezcla de auto y fr

En el ejemplo anterior, para calcular la anchura de la primera columna de la cuadrícula se tiene en cuenta la anchura de los elementos que se van a colocar en las celdas de dicha columna. Como la anchura máxima de dichos elementos es la del primer hijo, que tiene una anchura de `200px`, la columna se crea con dicha anchura, dejando para la otra columna el resto de anchura del contenedor.

MINMAX

En otras ocasiones queremos que la **anchura de una columna debe ser variable dentro de un determinado rango,** dependiendo de la anchura del contenedor. Para ello usaremos la función `minmax(tamaño_minimo, tamaño_maximo)`. Por ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: minmax(200px, 400px) 1fr;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de minmax()

En el ejemplo anterior, dependiendo de la anchura del contenedor, la primera columna tomará una anchura diferente, pero ésta siempre será como mínimo de `200px` y como máximo de `400px`.

Para especificar el valor mínimo NO se puede usar la unidad `fr`, aunque para especificar el tamaño máximo sí. Por ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: minmax(200px, 2fr) 1fr;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de minmax y fr

En el ejemplo anterior, dependiendo de la anchura del contenedor, la primera columna tomará un anchura diferente, pero ésta siempre será como mínimo de `200px` y como máximo de `2/3` de la anchura sobrante en el contenedor.

REPEAT (nº, tamaño)

Si vamos a crear varias columnas o filas adyacentes del mismo tamaño podemos usar la función `repeat(número, tamaño)` para acortar la especificación. Por ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: repeat(2, 200px);
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de repeat()

Auto-fill

La función anterior proporciona también un valor muy interesante para el primer parámetro, llamado `auto-fill`, que permite que **se calcule automáticamente el número de columnas de una determinada anchura que caben en el contenedor de la cuadrícula**. En el siguiente ejemplo se crearán **tantas columnas de `200px` de ancho como quepan en el contenedor, y la anchura sobrante no será utilizado**:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: repeat(auto-fill, 200px);
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de repeat() y auto-fill

Debemos tener en cuenta que si especificamos

```
grid-template-columns: repeat(auto-fill, 1fr);
```

se creará una única columna con todo el ancho del contenedor.

AMPLIAR : diferencia entre auto-fill y auto-fit.

Si combinamos el uso de `repeat(auto-fill, tamaño)` con la función `minmax(tamaño_minimo, 1fr)`, con el formato `repeat(auto-fill, minmax(tamaño_minimo, 1fr)`, podemos hacer que se creen tantas columnas como sea necesario, atendiendo al número de elementos, de como mínimo el tamaño específicado.

Si, dado el número de elementos, no es necesario crear más columnas, el espacio sobrante NO es repartido entre ellas, pero si, dado el número de elementos, se han creado todas las columnas posibles en la fila, el espacio sobrante se reparte entre ellas. En el siguiente ejemplo, **se crearán tantas columnas como para posicionar el número de elementos existente, de como mínimo `100px` de ancho, de manera que si el número de elementos es pequeño podrá quedar espacio sobrante**:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  overflow: hidden;
}
```

```
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de repeat(), auto-fill y minmax()

Auto-fit

Otra posibilidad muy parecida es usar el valor `auto-fit` **en vez de** `auto-fill`**. La diferencia radica, en que con** `auto-fit` **se crean tantas columnas como sea posible, independientemente del número de elementos que haya que posicionar en la cuadrícula.** Así si usamos `repeat(auto-fit, minmax(tamaño_minimo, 1fr)`, se crearán tantas columnas como sea posible, atendiendo a la anchura del contenedor de la cuadrícula y del tamaño mínimo indicado, de manera que **NO queden ningún espacio sobrante**, que será repartido entre las columnas creadas. Por ejemplo al especificar:

```
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

Se coge todo el ancho del contenedor de la cuadrícula y se divide entre `200px` y se crean tantas columnas como el cociente de dicha división. Posteriormente se coge el espacio sobrante y se reparte proporcionalmente entre dichas columnas. Todo ello independientemente del número de elementos que haya que posicionar en la cuadrícula. Veamos una demostración:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de auto-fit

Altura(FILAS):

En todos los ejemplos anteriores hemos especificado solamente la anchura de las columnas de la cuadrícula, de manera que las filas se han creado automáticamente cuando era necesario con la altura necesaria para mostrar los elementos. En el siguiente ejemplo, el tercer hijo es más alto que el resto, de manera que la fila que contenga dicho elemento es el que tendrá dicha altura:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

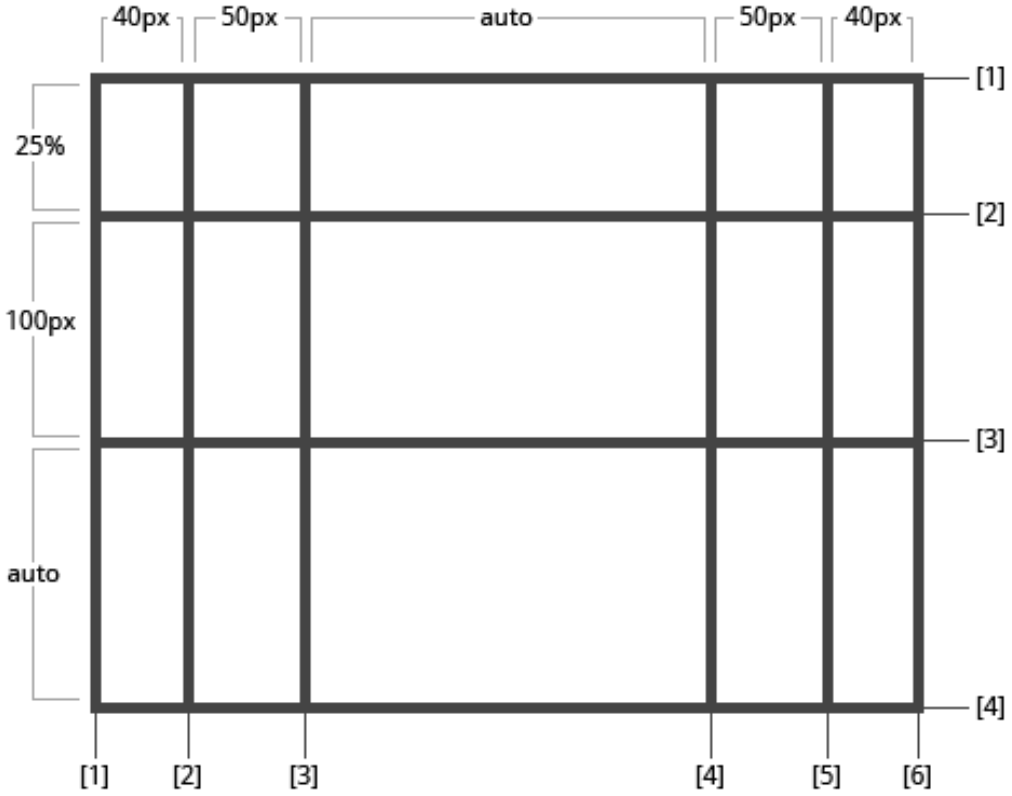
```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
.item:nth-child(3) {
  height: 200px;
}
```

> Demo de altura de un elemento

Mediante la propiedad `grid-template-rows` podemos indicar explícitamente la altura de las filas, usando valores similares a los usados con `grid-template-columns`. Veamos un ejemplo sencillo:

```
.container {
  grid-template-columns: 40px 50px auto 50px 40px;
  grid-template-rows: 25% 100px auto;
}
```

En el ejemplo anterior los elementos de la clase `container` crearán en su interior la siguiente cuadrícula:

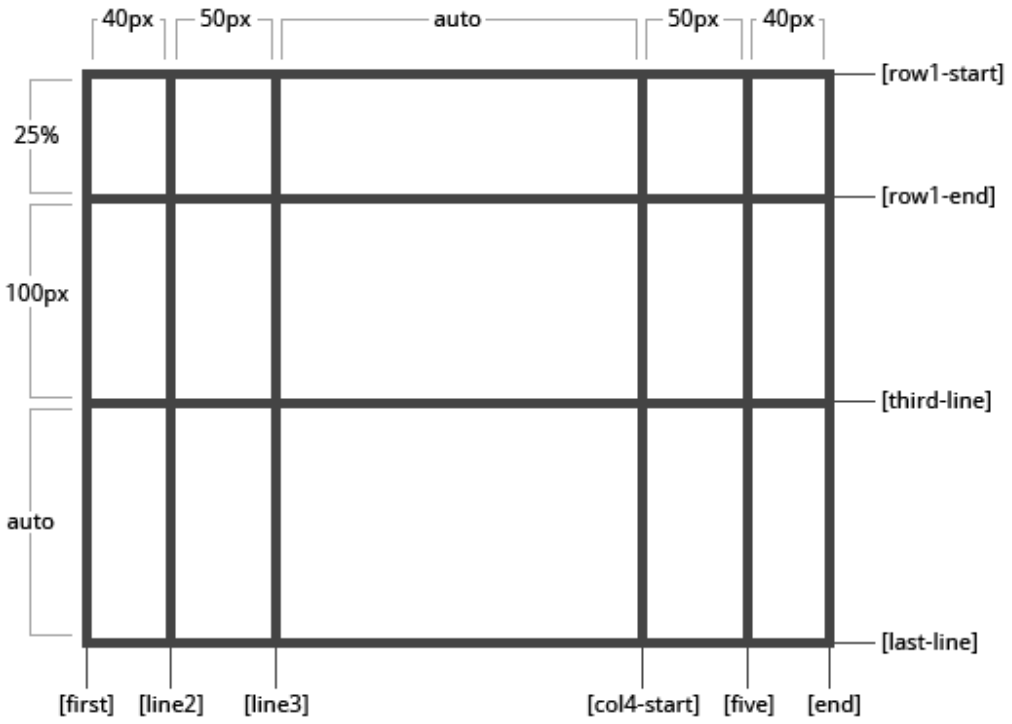


Nombres de las líneas de cuadrícula

Al especificar cada columna o cada fila, se crean también las correspondientes líneas de cuadrícula, que por defecto tomarán identificadores numéricos secuenciales (`1`, `2`, ...). Sin embargo al crear las columnas o las filas podemos también especificar el nombre que queremos dar a una determinada línea de cuadrícula, usando el formato `[nombre_linea]`, indicándolo delante o después del tamaño de la columna (dependiendo de si es la línea de cuadrícula anterior o posterior). Por ejemplo:

```
.container {
  grid-template-columns: [first line1] 40px [line2] 50px [line3] auto
                         [col4-start] 50px [five] 40px [end];
  grid-template-rows: [row1-start] 25% [row1-end] 100px [third-line] auto [last-
line];
}
```

Como resultado obtenemos la siguiente cuadrícula:



Una línea de cuadrícula puede recibir varios nombres. Como vemos en el ejemplo anterior, la primera línea de cuadrícula tiene dos nombres `first` y `line1` (este último no aparece en la imagen).

Otra opción es nombrar las líneas usando un determinado nombre de área seguido del sufijo `-start` o del sufijo `-end`, para delimitar las líneas entre las que está contenido un determinado área. De esta manera, posteriormente podremos usar simplemente el nombre de área para posicionar un determinado elemento de cuadrícula. Así, si por ejemplo definimos el contenedor de la siguiente manera:

```
.container {
  grid-template-columns: [header-start nav-start footer-start] 1fr
                         [nav-end content-start] 3fr [content-end header-end
footer-end];
  grid-template-rows: [header-start] 200px [header-end nav-start content-start]
auto
                      [nav-end content-end footer-start] 100px [footer-end];
}
```

a la hora de posicionar los elementos de cuadrícula podremos usar los nombres de área `header`, `nav`, `content` y `footer` para indicar el área en el que debe colocarse, como veremos cuando hablemos de las áreas de cuadrícula.

Separación entre columnas o entre filas

La propiedad `grid-column-gap` permite especificar la anchura del espacio entre columnas y `grid-row-gap` la altura del espacio entre filas. Dicho espacio es tenido en cuenta a la hora de hacer los cálculos con los tamaños especificados en la unidad `fr` para columnas o filas.

En inglés dicha separación se conoce tradicionalmente como *gutter*.

No debemos olvidar que la separación es siempre entre dos columnas o entre dos filas, por lo que NO se aplica en los lados de la cuadrícula, es decir, a la izquierda de la primera columna, a la derecha de la última columna, arriba de la primera fila ni debajo de la última fila.

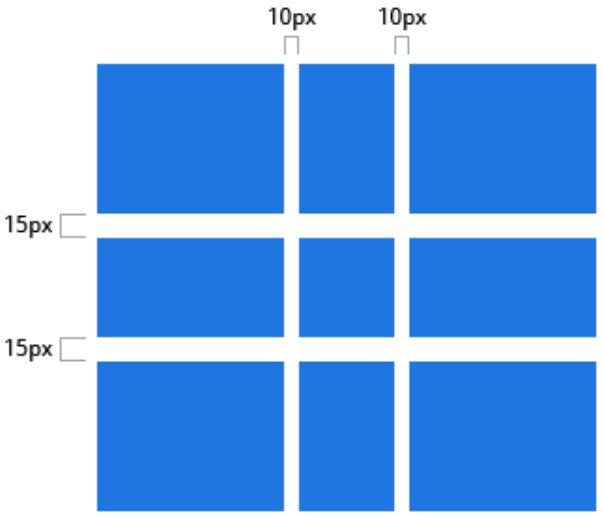
Otro aspecto importante es que **NO es posible colocar ningún elemento dentro de dicha separación**, sino que en dicha zona se mostrará simplemente el fondo del contenedor.

Propiedad	**grid-column-gap, grid-row-gap**
Valores	tamaño
Se aplica a	Contenedores de cuadrícula
Valor inicial	0
Descripción	Define el espacio entre columnas / filas.

Por ejemplo:

```
.container {
  grid-template-columns: 100px 50px 100px;
  grid-template-rows: 80px auto 80px;
  grid-column-gap: 10px;
  grid-row-gap: 15px;
}
```

En el ejemplo anterior se crea una cuadrícula donde las columnas se separan por `10px` y las filas por `15px`, tal y como se aprecia en la siguiente imagen:



Por su parte, la propiedad `grid-gap` es una propiedad *shorthand* de las dos anteriores. Recibe, en este orden, la separación entre filas y, opcionalmente, la separación entre columnas. Si no se especifica la separación entre columnas, ésta tomará también el valor de separación especificados para las filas.

Propiedad	**grid-gap, grid-row-gap**
Valores	<separacion_filas> <separacion_columnas>?
Se aplica a	Contenedores de cuadrícula

Propiedad	grid-gap, grid-row-gap
Valor inicial	0
Descripción	Define el espacio entre filas y entre columnas.

Por ejemplo:

```
.container {
  grid-template-columns: 100px 50px 100px;
  grid-template-rows: 80px auto 80px;
  grid-gap: 15px 10px;
}
```

4.Cuadrícula implícita

Como hemos visto en ejemplos anteriores, con sólo definir la cuadrícula en el contenedor los distintos elementos son posicionados automáticamente en ella aunque no hayamos especificado explícitamente dónde queremos colocar cada elemento. A este posicionamiento automático se le conoce como algoritmo de posicionamiento automático (*auto-placement algorithm*).

4.1 Algoritmo de posicionamiento automático

CSS nos permite incluso controlar cómo debe funcionar dicho algoritmo de posicionamiento automático, a través de la propiedad `grid-auto-flow`.

Propiedad	grid-auto-flow
Valores	row | column | row dense | column dense
Se aplica a	Contenedores de cuadrícula
Valor inicial	row
Descripción	Define cómo debe funcionar el algoritmo de posicionamiento automático de elementos en la cuadrícula.

El primer valor que podemos asignarle a esta propiedad es `row`, que hace que los elementos no colocados explícitamente se vayan posicionando automáticamente fila a fila, y dentro de cada fila, columna a columna (a la derecha y abajo), añadiendo columnas y filas conforme sea necesario. Se trata del valor por defecto, por lo que es el valor que ha sido usado en todos los ejemplos anteriores.

Al posicionar los elementos no colocados explícitamente, se respeta el orden en el que aparece en el archivo HTML, por lo que si un elemento anterior ha sido posicionado explícitamente respecto a la columna, se pueden producir huecos, es decir, celdas que no se rellenen. Es el valor por defecto. Por ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  grid-auto-flow: row;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
.item:nth-child(3) {
  grid-column: 1;
}
.item:nth-child(1) {
  grid-row: 3
```

```
}
```

> Demo de posicionamiento automático por defecto

En el ejemplo anterior, el primer elemento se coloca explícitamente en la tercera fila, por lo que el segundo elemento, que no es colocado explícitamente, se coloca en la primera fila primera columna. El tercer elemento se coloca explícitamente en la primera columna (no dice nada de la fila) , por lo que no se puede situar en la segunda columna de la primera fila, que es donde le correspondería. El cuarto elemento no es colocado explícitamente, pero como debe seguirse el orden indicado en el documento HTML, el cuarto elemento debe colocarse después del tercer elemento, por lo que se coloca en la siguiente columna a éste.

¿Por qué solo aparecen dos columnas?Alternativamente, podemos usar el valor `column`, de manera que los elementos no colocados explícitamente se vayan posicionando automáticamente columna a columna, y dentro de cada columna, fila a fila (abajo y a la derecha), añadiendo filas y columnas conforme sea necesario. Al posicionar cada elemento se respeta el orden en el que aparece en el archivo HTML, por lo que si un elemento anterior ha sido posicionado explícitamente respecto a la fila se pueden producir huecos, es decir, celdas que no se rellenen. Por ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  grid-auto-flow: column;
  overflow: hidden;
}
.item {
```

```
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
.item:nth-child(3) {
  grid-column: 1;
}
.item:nth-child(1) {
  grid-row: 3
}
```

> Demo de posicionamiento automático por columna

Si queremos evitar que se produzcan huecos en la cuadrícula podemos usar el valor `row dense`, en el que los elementos se vayan posicionando automáticamente fila a fila, y dentro de cada fila, columna a columna (a la derecha y abajo), añadiendo columnas y filas conforme sea necesario. Al posicionar cada elemento se rellenan los posibles huecos que existan en la cuadrícula, por lo que puede que NO se respete el orden en el que aparecen los elementos en el archivo HTML si hemos posicionado explícitamente la columna de un elemento anterior. Por ejemplo:

```
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  grid-auto-flow: row dense;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
.item:nth-child(3) {
  grid-column: 1;
}
.item:nth-child(1) {
  grid-row: 3
}
```

> Demo de posicionamiento row dense

Finalmente, también podemos usar `column dense`, de manera que los elementos se vayan posicionando automáticamente columna a columna, y dentro de cada columna, fila a fila (abajo y a la derecha), añadiendo filas y columnas conforme sea necesario. Al posicionar cada elemento se rellenan los posibles huecos que existan en la cuadrícula, por lo que puede que NO se respete el orden en el que aparecen los elementos en el archivo HTML si hemos posicionado explícitamente la fila de un elemento anterior. Por ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  grid-auto-flow: column dense;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
.item:nth-child(3) {
  grid-column: 1;
}
.item:nth-child(1) {
  grid-row: 3
```

```
}
```

> Demo de posicionamiento column dense

4.2 Tamaño de las filas y columnas creadas automáticamente

Como acabamos de ver, el algoritmo de posicionamiento automático puede crear automáticamente filas o columnas conforme sean necesarias. Igualmente, si posicionamos explícitamente un elemento en una celda que no existe, CSS modificará la cuadrícula para adaptarse a tal circunstancia, creando la filas y columnas que sean necesarias.

Por defecto las filas creadas automáticamente tendrán una altura `auto`, es decir, la necesaria para mostrar los elementos de dichas fila, atendiendo a la altura de dichos elementos. De forma similar ocurre con la anchura de las columnas creadas automáticamente.

Sin embargo, **mediante la propiedad `grid-auto-rows`, podemos especificar la altura con la que deben crearse dichas filas, y mediante la propiedad `grid-auto-columns`, la anchura con la que deben crearse dichas columnas.**

Propiedad	grid-auto-rows y grid-auto-columns
Valores	| auto
Se aplica a	Contenedores de cuadrícula
Valor inicial	auto
Descripción	Define la altura / anchura con la que deben definirse las filas / columnas creadas automáticamente en la cuadrícula.

La longitud se puede expresar en `px`, en `%` respecto al tamaño del contenedor o usando la unidad `fr`. Por ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  grid-auto-rows: 300px;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
.item:nth-child(3) {
  height: 200px;
}
```

> Demo de grid-auto-rows

En el ejemplo anterior las filas creadas automáticamente tienen una altura de `300px`. Por defecto los elementos se estiran para ocupar la altura de la fila, pero el tercer elemento no, ya que se le ha indicado explícitamente una altura de `200px`.

5.Posicionamiento de elementos de cuadrícula

Por defecto los elementos de cuadrícula se colocan ocupando una celda de la cuadrícula atendiendo al orden en el que aparecen en el documento HTML. De hecho, en los ejemplos anteriores NO hemos especificado explícitamente en que celda/s debía colocarse cada elemento hijo directo del contenedor, por lo que se ha usado el posicionamiento por defecto.

Sin embargo, para cada elemento de cuadrícula podremos especificar explícitamente entre qué líneas de cuadrícula debe posicionarse dicho elemento.

Las propiedades `grid-column-start`, `grid-column-end`, `grid-row-start` y `grid-row-end` nos permiten indicar, respectivamente, la línea de columna inicial, la línea de columna final, la línea de fila inicial y la línea de fila final. El elemento de cuadrícula se situará en las celdas contenidas entre dichas cuatro líneas, que puede corresponder a 1 o más celdas.

Propiedad	**grid-column-start, grid-row-start**
Valores	| <nombre_linea> | auto
Se aplica a	Elemento de cuadrícula
Valor inicial	auto
Descripción	Define la columna/fila inicial del elemento dentro de la cuadrícula.

Propiedad	**grid-column-end, grid-row-end**
Valores	| <nombre_linea> | auto | span | span <nombre_linea>
Se aplica a	Elemento de cuadrícula
Valor inicial	auto
Descripción	Define la columna/fila final del elemento dentro de la cuadrícula.

CSS también nos proporciona las propiedades *short-hand* `grid-column` y `grid-row`, que usando la sintáxis `inicio / fin` permiten posicionar los elementos en la rejilla de una manera algo más abreviada.

Propiedad	**grid-column, grid-row**
Valores	| <nombre_linea> | auto / | <nombre_linea> | auto | span | span <nombre_linea>
Se aplica a	Elemento de cuadrícula
Valor inicial	auto / auto
Descripción	Define las columnas/filas inicial y final del elemento dentro de la cuadrícula.

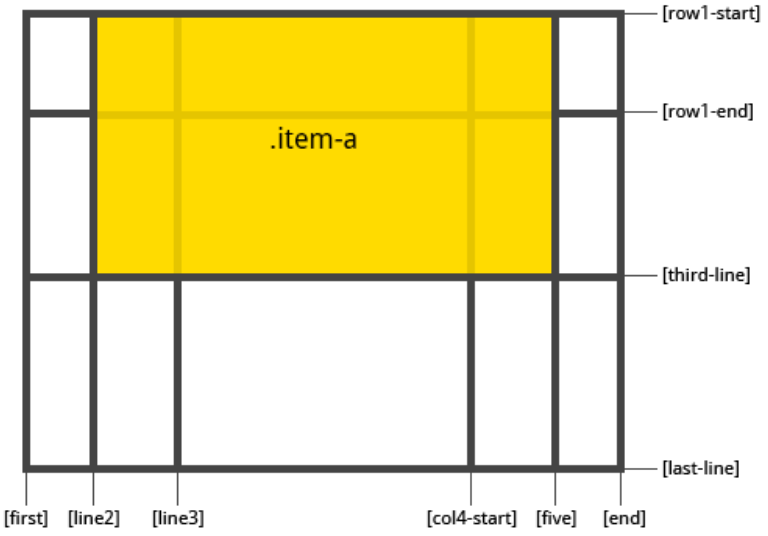
Como valor para estas propiedades podemos especificar:

- Un entero, correspondiente al **identificador numérico** de la línea. La primera línea tiene el identificador `1`. Por ejemplo `grid-column-start: 2`.
- Un nombre de línea. Por ejemplo `grid-column-start: fist`
- `span <número de columnas o filas>` o `span <nombre_linea>`: Se usa en las propiedades `grid-column-end` / `grid-row-end` para indicar cuántas columnas/filas o hasta que línea final debe expandirse el posicionamiento del elemento respecto a la línea inicial. Por ejemplo `grid-column-end: span 3` indica que el elemento debe ocupar tres **columnas** a partir de la línea inicial establecida mediante `grid-column-start`.
- `auto`: Posicionamiento automático con `span 1`.

Siguiendo la cuadrícula establecida en un ejemplo anterior, si establecemos:

```
.item-a {
  grid-column-start: 2;
  grid-column-end: five;
  grid-row-start: row1-start
  grid-row-end: 3
}
```

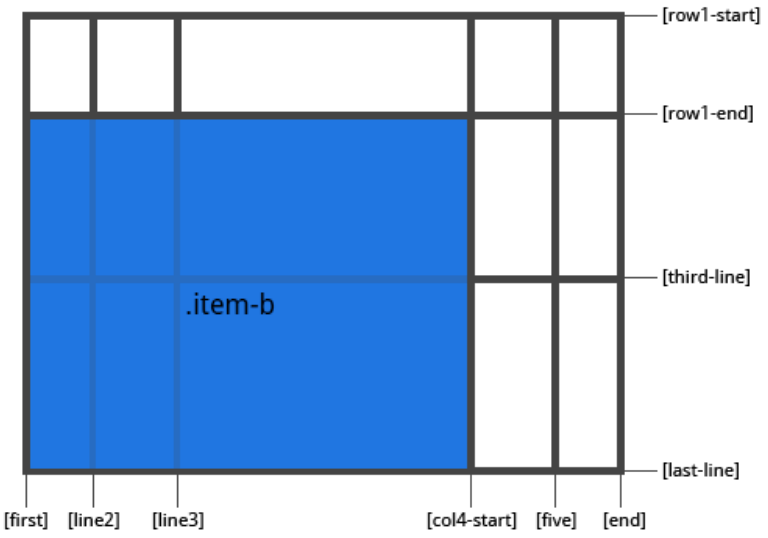
el elemento de la clase `item-a` se posicionará de la siguiente manera:



Por otra parte, si establecemos:

```
.item-b {
  grid-column-start: 1;
  grid-column-end: span col4-start;
  grid-row-start: 2
  grid-row-end: span 2
}
```

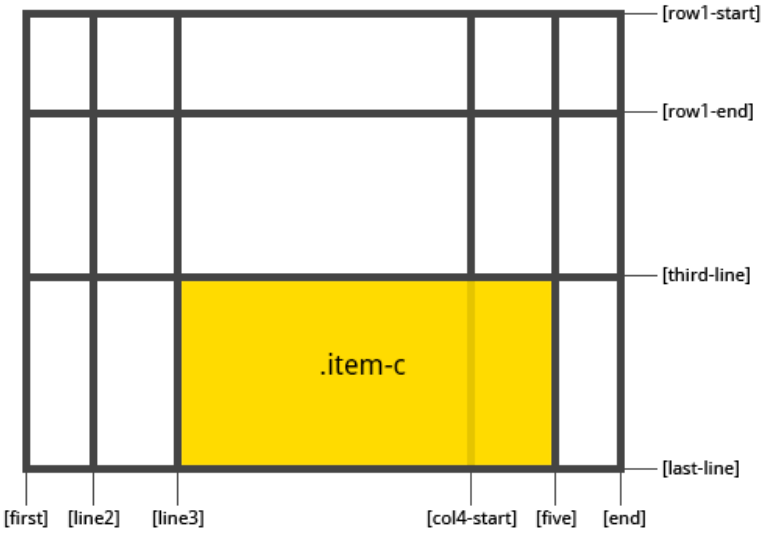
el elemento de la clase `item-b` se posicionará de la siguiente manera:



Finalmente, si posicionamos el elemento de la clase `item-c` usando las propiedades *short-hand*:

```
.item-c {
  grid-column: 3 / span 2;
  grid-row: third-line / 4;
}
```

se situará de la siguiente manera:



6. Áreas de cuadrícula

Trabajar con líneas de cuadrícula es algo engorroso, como podemos apreciar en los ejemplos anteriores. Sin embargo, CSS nos brinda una nueva posibilidad, consistente definir en la cuadrícula las áreas que la conforman (y qué celdas ocupan) e indicar para cada elemento en qué área debe posicionarse.

Definición de áreas

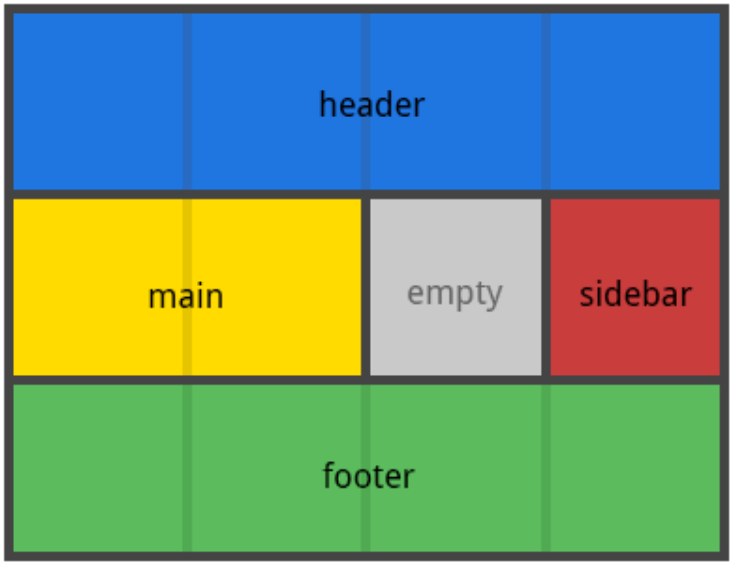
La propiedad `grid-template-areas` permite indicar a qué área de cuadrícula pertenece cada celda de la misma.

Propiedad	grid-template-areas
Valores	" | . | none | ..." "..."
Se aplica a	Contenedores de cuadrícula
Valor inicial	none
Descripción	Define a qué área pertenece cada celda de la cuadrícula.

f

```
.container {
  grid-template-columns: 50px 50px 50px 50px;
  grid-template-rows: auto;
  grid-template-areas:
    "header header header header"
    "main main . sidebar"
    "footer footer footer footer";
}
```

En el ejemplo anterior se define un contenedor de cuadrícula que posee cuatro columnas y tres filas. Para cada celda se indica a qué área pertenece: `header`, `main` `sidebar`, `footer` o ninguna. La cuadrícula resultante tiene las siguientes áreas:



Las líneas de cuadrícula de comienzo y de final de cada área son nombradas automáticamente como `area-start` y `area-end`, lo que implica que una línea de cuadrícula puede tener varios nombres, como al línea situada más a la izquierda de la cuadrícula anterior, que tendrá los nombres `header-start`, `main-start`, y `footer-start`.

Posicionamiento de un elemento en un área

La propiedad `grid-area` permite especificar el área de la cuadrícula en la que debe posicionarse un elemento, ya sea indicando el nombre del área o la fila / columna inicial y la fila / columna final.

Propiedad	**grid-area**
Valores	<nombre_area> | / / /
Se aplica a	Elementos de cuadrícula
Valor inicial	auto / auto / auto / auto
Descripción	Posiciona el elemento en el área indicada.

Como valor de la propiedad podemos indicar:

- El nombre del área en el que se quiere posicionar el elemento.
- **Cuatro valores correspondientes (por este orden) a la fila inicial, la columna inicial, la fila final y la columna final, actuando como propiedad short-hand de `grid-row-start`, `grid-column-start`, `grid-row-end` y `grid-column-end`, respectivamente.**

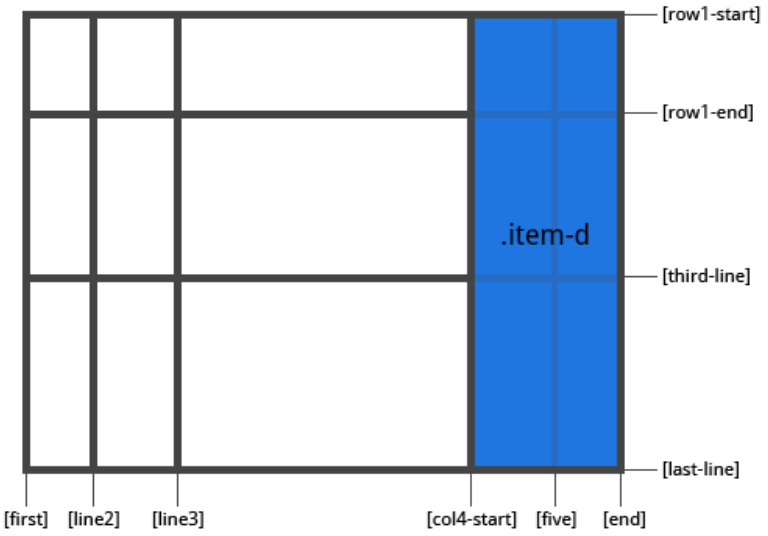
Así si establecemos:

```
.item-d {
  grid-area: header;
}
```

que sería similar a establecer:

```
.item-d {
  grid-area: 1 / col4-start / last-line / 6;
}
```

el elemento se posicionaría de la siguiente manera:



7.Propiedades de alineación de elementos en las celdas

Como hemos visto en los ejemplos anteriores, por defecto los elementos ocuparán todo el ancho y todo el alto de la celda en la que se coloca. Sin embargo podemos modificar dicho comportamiento estableciendo la alineación horizontal y vertical del elemento respecto a la celda que lo contiene.

Alineación horizontal

La propiedad `justify-items` permite establecer **para el contenedor cómo se deben alinear horizontalmente todos los elementos dentro de cada celda.**

Propiedad	justify-items
Valores	start | end | center | stretch
Se aplica a	Contenedor de cuadrícula
Valor inicial	stretch
Descripción	Alinea horizontalmente los elementos dentro de cada celda.

Los valores posibles son: `start` (a la izquierda), `end` (a la derecha), `center` (centrado) y `stretch` (el elemento se estira para ocupar todo el ancho de la celda, es el valor por defecto). Por ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  justify-items: center;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de justify-items

En el ejemplo anterior hemos indicado una anchura concreta a los elementos, por lo que dependiendo del tamaño del contenedor puede que las celdas sean más anchas que los elementos. Hemos indicado que en ese caso los elementos se muestran centrados horizontalmente respecto a la celda en la que están contenidos.

Si queremos que un determinado elemento se alinee horizontalmente de una forma diferente podemos usar **en dicho elemento** la propiedad `justify-self`, que recibe los mismos valores que `justifiy-items`:

Propiedad	**justify-self**
Valores	start | end | center | stretch
Se aplica a	Elemento de cuadrícula
Valor inicial	stretch
Descripción	Alinea horizontalmente dicho elemento dentro de su celda, sobrescribiendo lo especificado por la propiedad justify-items del contenedor.

En el siguiente ejemplo hacemos que uno de los elementos se alinee de una forma distintos a la establecida para todo los elementos en el contenedor:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  justify-items: center;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
.item:nth-child(2) {
  justify-self: stretch;
}
```

> Demo de align-self

Alineación vertical

La propiedad `align-items` permite establecer **para el contenedor cómo se deben alinear verticalmente todos los elementos dentro de cada celda.**

Propiedad	align-items
Valores	start | end | center | stretch

Propiedad	align-items
Se aplica a	Contenedor de cuadrícula
Valor inicial	stretch
Descripción	Alinea verticalmente los elementos dentro de cada celda.

Los valores posibles son: `start` (arriba), `end` (abajo), `center` (centrado) y `stretch` (el elemento se estira para ocupar todo el alto de la celda, es el valor por defecto).

Si queremos que un determinado elemento se alinee verticalmente de una forma diferente podemos usar **en dicho elemento** la propiedad `align-self`, que recibe los mismos valores que `align-items`:

Propiedad	align-self
Valores	start | end | center | stretch
Se aplica a	Elemento de cuadrícula
Valor inicial	stretch
Descripción	Alinea verticalmente dicho elemento dentro de su celda, sobrescribiendo lo especificado por la propiedad align-items del contenedor.

8.Propiedades de alineación de la cuadrícula respecto al contenedor

Algunas veces el tamaño total de la cuadrícula especificada en el contenedor es menor que el tamaño del propio contenedor, al NO haber utilizado unidades flexibles como `fr`, `auto-fill`, o `auto-fit`, sino unidades fijas como `px` a la hora especificar los tamaños de columnas o filas. En ese caso es posible indicar cómo debe alinearse la cuadrícula dentro del contenedor, tanto horizontal como verticalmente.

Alineación horizontal

La propiedad `justify-content` permite establecer **para el contenedor** cómo se debe alinear horizontalmente la cuadrícula dentro de él.

Propiedad	justify-content
Valores	start | end | center | stretch | space-around | space-between | space-evenly
Se aplica a	Contenedor de cuadrícula
Valor inicial	start
Descripción	Alinea horizontalmente la cuadrícula dentro del contenedor

Los valores posibles son:

- `start` (a la izquierda), `end` (a la derecha), `center` (centrada),

- `stretch`: Redimensiona proporcionalmente las columnas para que la cuadrícula ocupe toda la anchura del contenedor.
- `space-around`: Reparte el espacio sobrante como separación entre columnas, de manera que la primera columna tiene a su izquierda la mitad de separación y la última columna tiene a su derecha la mitad de separación.
- `space-between`: Reparte el espacio sobrante como separación entre columnas, de manera que la primera columna esté lo más a la izquierda posible y la última columna lo más a la derecha posible.
- `space-evenly`: Reparte el espacio sobrante como separación entre columnas, de manera que todas las columnas tienen entre sí la misma separación, incluyendo la separación a la izquierda de la primera columna y la separación a la derecha de la última columna.

Veamos un ejemplo:

```html
<section class="grid">
  <div class="item">item1</div>
  <div class="item colocado">item2</div>
  <div class="item">item3</div>
  <div class="item">item4</div>
  <div class="item">item5</div>
</section>
```

```css
.grid {
  width: 80%;
  margin: 10px auto;
  background-color: tomato;
  display: grid;
  grid-gap: 10px;
```

```
  grid-template-columns: 200px 200px;
  justify-content: center;
  overflow: hidden;
}
.item {
  text-align: center;
  background-color: #D73B38;
  color: #ffffff;
  padding: 10px;
  border-radius: 5px;
}
```

> Demo de justify-content

Alineación vertical

La propiedad `align-content` permite establecer **para el contenedor** cómo se debe alinear verticalmente la cuadrícula dentro de él.

Propiedad	align-content
Valores	start | end | center | stretch | space-around | space-between | space-evenly
Se aplica a	Contenedor de cuadrícula
Valor inicial	start
Descripción	Alinea verticalmente la cuadrícula dentro del contenedor

Los valores posibles son:

- `start` (arriba), `end` (abajo), `center` (centrada),
- `stretch`: Redimensiona proporcionalmente las filas para que la cuadrícula ocupe toda la altura del contenedor.
- `space-around`: Reparte el espacio sobrante como separación entre filas, de manera que la primera fila tiene arriba la mitad de separación y la última fila tiene abajo la mitad de separación.
- `space-between`: Reparte el espacio sobrante como separación entre filas, de manera que la primera fila esté lo más arriba posible y la última fila lo más abajo posible.
- `space-evenly`: Reparte el espacio sobrante como separación entre filas, de manera que todas las filas tienen entre sí la misma separación, incluyendo la separación arriba de la primera fila y la separación abajo de la última fila.

Demo de cuadrícula implícita y alineación

Veamos una demostración:

Demo de cuadrícula implícita y alineación

Demo de áreas de cuadrícula y posicionamiento

(pdte)

Generadores de cuadrículas

https://www.layoutit.com/grid

Referencias

https://css-tricks.com/snippets/css/complete-guide-grid/

https://escss.blogspot.com/2017/04/css-grid-layout-tras-soporte-universal.html

http://codepen.io/collection/XQKoYq/

https://css-tricks.com/difference-explicit-implicit-grids/

https://www.youtube.com/watch?v=5GGhOJWlVfM

https://gridbyexample.com/video/

https://medium.com/@andybarefoot/stretching-the-grid-5-fun-ways-to-use-css-grid-5931166f467f

https://www.youtube.com/watch?v=kqB7XD0d0gA&list=PLM-Y_YQmMEqBxmylkI5WJn9ouUxWlJNOW